Real Estate Investment Advisor

Predicting Property Profitability & Future Value

Project documentation: methodology, findings and use cases

Domain: Real Estate / Investment / Financial Analytics

Stack: Python, Pandas, Scikit-learn, XGBoost, MLflow, Streamlit, Plotly, Matplotlib/Seaborn, SciPy

Dataset: india_housing_prices.csv, 35,359 listings, 42 cities

Classification: *Is this a Good Investment?* | Regression: *Estimated price after 5 years*

1. Executive summary

This project builds a decision-support tool for property investors. For any property it answers two questions:

1. **Classification:** *Is this a Good Investment?* (yes/no, with a confidence score)
2. **Regression:** *What will the property be worth in 5 years?* (point estimate and 90% range)

The work covers the full workflow: data cleaning, feature engineering, 24 statistically tested EDA analyses (the 20 questions in the brief plus the 4 Step-2 analyses), three model families per task, MLflow experiment tracking and a model registry, and a six-page Streamlit application.

Headline results (held-out test set, 7,072 properties)

Task	Champion	Key metrics
Good Investment (classification)	XGBoost	Accuracy 0.992, Precision 0.989, Recall 0.987, F1 0.988, ROC-AUC 0.9996
Price after 5 years (regression)	XGBoost	RMSE 1.73 lakhs, MAE 1.34 lakhs, R^2 0.9999, MAPE 0.55%
Naive baseline (fixed 8% growth)	n/a	RMSE 23.33 lakhs, MAE 16.43 lakhs, R^2 0.988

Important context (please read). The dataset contains no price history, so both targets are *rule-derived* from the features using documented assumptions (Section 4.3). The models are therefore very accurate at reproducing those rules, which is what the brief's design implies, but this does **not** prove they can forecast the real market. In addition, the data itself behaves like simulated data: price and size are uniformly distributed and statistically unrelated to city, size, BHK, amenities or infrastructure (a model predicting today's price from all other features scores $R^2 = -0.15$). Section 6 explains how to read the results honestly, and Section 8 lists what to do with real data.

2. Problem statement and business use cases

Develop an application that helps investors assess long-term returns on real estate.

Investors: *screen listings quickly with a transparent scorecard and a 5-year value estimate.* **Home buyers:** shortlist high-return properties in developing areas by filtering on city, budget, BHK and more. ***Real-estate companies:*** *automate investment analysis for every listing.* **Platforms:** increase customer trust through data-backed, explainable predictions (scorecard, confidence, feature importance).

3. Data

Source: `india_housing_prices.csv`, 35,359 listings x 23 columns, 20 states, 42 cities, 500 locality labels.

3.1 Data-quality findings and how each was handled

Check	Finding	Action
Missing values	None	Median/mode imputation logic kept in the pipeline for future extracts
Duplicates	0 duplicate rows, 0 duplicate IDs	Drop-duplicate step retained
Impossible values	None (size, price, BHK, year all valid)	Validation step retained
Age_of_Property	Equals 2025 − Year_Built for every row	Re-derived as the single source of truth
Floor_No > Total_Floors	**16,647 rows (47.1%)**, which is physically impossible	Total_Floors raised to Floor_No; row flagged `Floor_Data_Repaired` (dropping would remove half the data)
Price_per_SqFt	Supplied in lakhs, rounded to 2 decimals; **331 rows are 0.00**	Recomputed exactly in rupees: `Price_per_SqFt_INR = Price_in_Lakhs x 100,000 / Size`
Amenities	325 spellings of the same 5 amenities (only the order differs)	Canonicalised to 31 sorted combinations; 5 binary flags and a count created

Check	Finding	Action
Outliers (IQR 1.5x)	Price/sq ft: 2,808 high outliers (7.9%); price and size: none	Flagged and kept (valid small-home/high-price combinations); winsorised at the 99th percentile for modelling
Locality	`Locality_1 … Locality_500` appear in every city (about 71 listings each)	Treated as anonymous labels, not physical neighbourhoods
Missing columns from the brief	No crime rate, no infrastructure score, no RERA, no date, no "Sold" status	Proxies built or analyses adapted (see Section 5.1)

4. Methodology

Pipeline: `data_cleaning.py` → `feature_engineering.py` → `eda.py` → `train_models.py` (MLflow) → `evaluate.py` → Streamlit app. One command runs everything: `python src/run_pipeline.py`.

4.1 Preprocessing

Clean and validate as in Section 3.1. **Scaling:** `StandardScaler` on numeric features (inside the sklearn pipeline, fitted on training data only). * **Encoding:** `OneHotEncoder(handle_unknown="ignore")` for City, Property_Type, Furnished_Status, Public_Transport_Accessibility, Facing, Owner_Type, Availability_Status. Parking and Security become 0/1 flags. Locality (500 anonymous labels) and State (fully determined by City) are not used as model inputs.

4.2 Feature engineering

Feature	Definition
Price_per_SqFt_INR	Price (lakhs) x 100,000 / size
PPSF_vs_City_Median	Price per sq ft divided by the city's median (value signal)
School_Density_Score	Nearby schools / 10
Infrastructure_Score	Mean of school score, hospital score and transport score (0, 0.5, 1)
Has_Playground/Gym/Garden/Pool/Clubhouse, Amenity_Count	Parsed from the amenity text
Has_Parking, Has_Security	Yes/No → 1/0
Floor_Ratio	Floor_No / Total_Floors
Size_per_BHK	Size / BHK
Log_Price	ln(price): makes the multiplicative 5-year target additive for linear models

4.3 Target creation (documented assumptions, all in `src/config.py`)

Regression target, `Future_Price_5Y_Lakhs = Price x (1 + r)^5`**.** The growth rate *r* starts at the brief's 8% and varies by location, type and features (the brief's "by location/type" and "feature-based" options):

Factor	Adjustment to r
Tier-1 city (Mumbai, New Delhi, Bangalore, Chennai, Hyderabad, Kolkata, Pune, Ahmedabad, Gurgaon, Noida)	+0.5 pts
Independent House / Villa	+0.3 / +0.6 pts
Infrastructure score	(score − 0.5) x 4 pts
Under construction	+0.5 pts
Built ≤ 5 years ago / older than 25 years	+0.5 / −0.5 pts
Bounds	5% to 12% per year

The average assumed growth is 8.70% a year (about +52% over 5 years).

Classification target, `Good_Investment`**.** A weighted multi-factor score (brief: "Price vs median", "Price per sq ft", "multi-factor score"). A property is *Good* if it scores **at least 6 of 9** points:

Criterion	Points
Price per sq ft at or below the city median	2
Projected growth ≥ 8.5% a year	2
3+ BHK	1
Ready to move	1
High public-transport accessibility	1
Infrastructure score ≥ 0.6	1
Security available	1

This yields 32.4% "Good" properties (a realistic, selective class balance). The growth rate itself is **not** a model input; the models see only what an investor can enter.

4.4 Modelling and evaluation

One shared stratified 80/20 split (28,287 train / 7,072 test); the test set is used only for final reporting. **Classification:** Logistic Regression, Random Forest, XGBoost. **Regression:** Linear Regression, Random Forest, XGBoost (all trained on log-price via `TransformedTargetRegressor`). *Tree models tuned with* `RandomizedSearchCV` *(3-fold). The champion is chosen by* ***cross-validation*** *score (ROC-AUC / RMSE), not by test score.* Metrics: Accuracy, Precision, Recall, F1, ROC-AUC (classification); RMSE, MAE, R^2, MAPE (regression), plus a fixed-8% naive baseline. ***Prediction interval:*** *the 5th and 95th percentiles of test-set relative errors give a 90% range (−1.13% to +1.11% of the estimate).* **Feature importance:** permutation importance on the original columns.

4.5 MLflow

Experiment `Real_Estate_Investment_Advisor`: 7 runs (6 models plus the baseline). Each run logs parameters, CV and test metrics, diagnostic plots, and the model with its signature. Champions are registered as `GoodInvestmentClassifier` and `FuturePriceRegressor` (alias `champion`). Open the UI with `mlflow ui --backend-store-uri sqlite:///mlflow.db`. A CSV export of all runs is in `outputs/reports/mlflow_runs_export.csv`.

5. Findings

5.1 Exploratory data analysis: answers to the 20 questions (+4 Step-2 analyses)

Every answer is computed from the data with a significance test and an effect size (ANOVA η^2, chi-square, Welch t-test, Pearson/Spearman). Charts are in `outputs/eda/`; the app's **EDA** page shows them interactively.

#	Question	Answer
1	Distribution of prices	Almost perfectly uniform from ₹10 L to ₹500 L (median ₹254.8 L, skew +0.01). Real prices are normally right-skewed.
2	Distribution of sizes	Uniform from 500 to 5,000 sq ft (median 2,749).
3	Price/sq ft by property type	Apartment ₹13,281, Villa ₹13,043, Independent House ₹13,009: a 2.1% gap, not significant (p = 0.22).
4	Size vs price	r = 0.000: size explains 0% of price. Homes under 1,000 sq ft average ₹255.7 L; 4,000-5,000 sq ft average ₹254.8 L.
5	Outliers	2,808 price/sq ft outliers (7.9%), all small homes (median 792 sq ft). None in price or size.
6	Price/sq ft by state	Karnataka highest (₹13,633), Kerala lowest (₹12,463); not significant (p = 0.84).
7	Average price by city	Jaipur highest (₹265.1 L), Kochi lowest (₹246.5 L); not significant (p = 0.91). Mumbai ranks 36th of 42.
8	Median age by locality	Locality medians range 13-25 years (oldest Locality_127, newest Locality_19). Locality names repeat across cities.
9	BHK across cities	Each BHK is about 20% of listings in every city (chi-square p = 0.50).

#	Question	Answer
10	Top-5 locality trends	Top-5 by price/sq ft: Locality_310, 123, 265, 133, 36. No date column exists; by build-year cohort there is no consistent pattern. Today vs 5-year projection is charted.
11	Correlations	Only built-in relationships are strong (price/sq ft vs price and size; infrastructure score vs schools/hospitals; floors). All others have $\|r\| \leq 0.01$.
12	Schools vs price/sq ft	$r = -0.003$; no relationship.
13	Hospitals vs price/sq ft	$r = -0.011$; negligible.
14	Price by furnishing	Furnished ₹256.2 L vs Unfurnished ₹253.2 L; not significant ($p = 0.24$).
15	Price/sq ft by facing	West ₹13,277 vs South ₹13,043; not significant ($p = 0.60$).
16	Owner types	Owner 11,847; Builder 11,845; Broker 11,667 (evenly split).
17	Availability	Ready to Move 17,721 (50.1%); Under Construction 17,638 (49.9%). No "Sold" status exists.
18	Parking vs price	₹254.6 L with parking vs ₹254.3 L without; not significant ($p = 0.83$).
19	Amenities vs price/sq ft	Differences of −0.9% to −2.5% (slightly lower with amenities); too small to matter.
20	Public transport	No relationship with price/sq ft ($\eta^2 = 0.0002$). "Good Investment" share rises 8% → 23% → 67% (Low → Medium → High), but this is built into the label rule, not discovered market behaviour.
21	Price trends by city	Price by build-year cohort moves between ₹252 L and ₹258 L with no significant pattern; 5-year projections shown for the 10 priciest cities.
22	Area vs investment return	No correlation between size and projected ROI % ($r = +0.006$).
23	Crime rate vs Good Investment	No crime column; Security used as proxy (40.4% good with security vs 24.5% without), an effect built into the rule.
24	Infrastructure vs resale value	$r = -0.006$ with price/sq ft; no infrastructure premium in this data.

Main EDA conclusion: in this dataset price is essentially independent of location, size, amenities and infrastructure. Investment quality can only be differentiated through the rule-based growth and scorecard assumptions.

5.2 Model results (test set)

Classification (selected by 3-fold CV ROC-AUC)

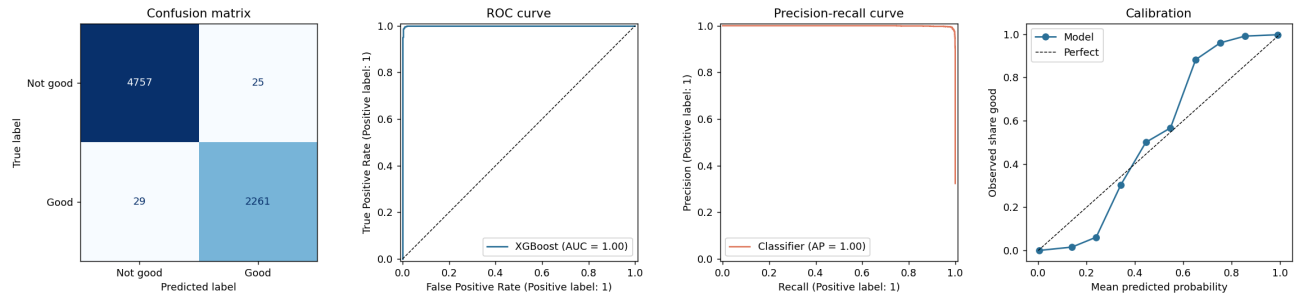
Model	CV AUC	Accuracy	Precision	Recall	F1	Test AUC
Logistic Regression	0.952	0.879	0.829	0.790	0.809	0.952
Random Forest	0.989	0.952	0.950	0.897	0.923	0.990
XGBoost	**0.999**	**0.992**	**0.989**	**0.987**	**0.988**	**0.9996**

Regression (selected by 3-fold CV RMSE)

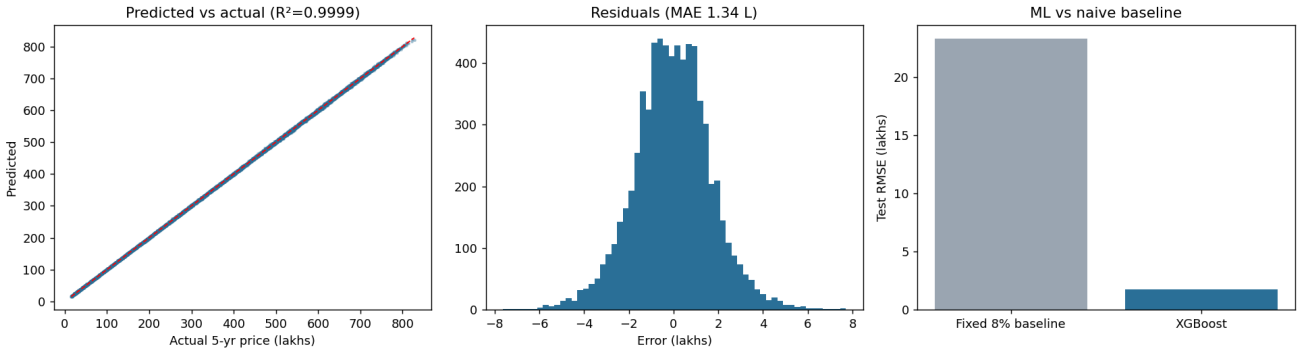
Model	CV RMSE (L)	Test RMSE (L)	MAE (L)	R^2	MAPE
Baseline: fixed 8%	n/a	23.33	16.43	0.9883	4.20%
Linear Regression	3.06	3.08	2.31	0.9998	0.60%
Random Forest	7.56	6.93	4.96	0.9990	1.42%
XGBoost	**1.90**	**1.73**	**1.34**	**0.9999**	**0.55%**

XGBoost cuts RMSE by 92.6% versus the fixed-8% baseline because it learns the location, type and infrastructure adjustments. No sign of overfitting: train and test errors are close (train RMSE 1.58 vs test 1.73).

What drives the models. Classifier: Infrastructure_Score, PPSF_vs_City_Median, Has_Security, Public_Transport_Accessibility, BHK and Availability_Status, which are exactly the criteria in the scorecard. Regressor: current price dominates, followed by Infrastructure_Score, Age, Availability and Property_Type, the growth-rate drivers.



Classifier diagnostics



Regressor diagnostics

6. Reading the results honestly

Two checks, run in `evaluate.py`, show why the high scores should be interpreted carefully.

1. **Signal test.** An XGBoost model trying to predict **today's** price from every other feature (excluding price-derived ones) scores **$R^2 = -0.15$** (RMSE 151.6 L against a price standard deviation of 141.6 L). The features contain no information about price.
2. **Ablation.** Removing the four price-based features (price, log price, price/sq ft, price vs city median) from the classifier lowers ROC-AUC from 0.9996 to **0.957** and accuracy from 0.992 to 0.886. The model leans on the price features that define part of the label.

Interpretation. The models reproduce the business rules very accurately, and that is the scope of the claim. They are not evidence of ability to predict real appreciation.

7. Streamlit application

Run: `streamlit run app/streamlit_app.py`

Page	What it does
Dashboard	KPIs; location heat-map (State or City x Property Type) for price/sq ft, price, % good investment or ROI; top cities by good-investment share; price path chart
Investment Advisor	Form for all property details. Returns "Good Investment / Not" with **confidence** and a probability gauge; **5-year estimated price** with 90% range, gain and implied growth; price-path chart; a **scorecard** showing which rules the property meets; price/sq ft vs the city median; **feature importance**
Property Finder	Filter by state, city, type, BHK, price, size, availability, transport, furnishing, owner type, parking, security and model probability; rank by probability, ROI, price or price/sq ft; scatter and city charts; CSV download
EDA	All 24 analyses with chart, auto-generated tested finding and supporting table
Model Insights	Test metrics, MLflow comparison tables, diagnostics, feature importance, honesty checks and assumptions
About	Methodology and data caveats

The app loads the exported `.joblib` models, so it needs no MLflow server. It was tested headlessly (all pages, all 24 EDA questions, advisor submissions and finder filters) with Streamlit's `AppTest`.

Deployment (Streamlit Community Cloud): push the repository to GitHub, create a new app with main file `app/streamlit_app.py`; the lean `app/requirements.txt` is used automatically.

8. Limitations and recommended next steps

1. **Replace assumed targets with real history.** Use dated sale or listing prices (registry or portal data) to measure true appreciation per city and locality, then retrain.
2. **Add the missing signals** the brief mentions: crime rate (e.g., NCRB by city), RERA registration, real infrastructure/connectivity scores, latitude/longitude, listing date.
3. **Use real locality identifiers** so locality-level analysis and heat-maps on a map become possible.
4. **Validate out of time:** train on older periods and test on newer ones; add calibration and fairness checks before any production use.
5. Treat all outputs as **educational, not financial advice**.

9. Deliverables map

Brief deliverable	Location
Cleaned and processed dataset	`data/processed/cleaned_housing.csv`, `featured_housing.csv`
Python scripts for EDA, model training, evaluation	`src/data_cleaning.py`, `feature_engineering.py`, `eda.py` (+ `eda_questions.py`), `train_models.py`, `evaluate.py`, `run_pipeline.py`
Streamlit application	`app/streamlit_app.py`
Project documentation	this file; also `README.md`
Video explaining the project	script and shot list in `docs/VIDEO_SCRIPT.md`
MLflow tracking	`mlflow.db` (re-created by training), export in `outputs/reports/mlflow_runs_export.csv`

10. Likely evaluation questions

Why is accuracy so high? The targets are rule-derived from the features, so a good model recovers the rules. The ablation (AUC 0.957 without price features) and the signal test (R^2 −0.15) put this in context.

Why not predict current price? The features carry no information about it in this data ($R^2 \approx 0$). Future price is modelled as current price x growth, so the investor supplies the asking price.

How was leakage handled? The growth rate (used to build both targets) is not a model input. The 99th-percentile caps and city medians come from the data pipeline, and the champion is selected on cross-validation, not test results. Price-based features are acknowledged as part of the label rule.

Why XGBoost? Best CV ROC-AUC (0.9993) and lowest CV RMSE (1.90 L), fast to train, and a small model file (0.3 MB).

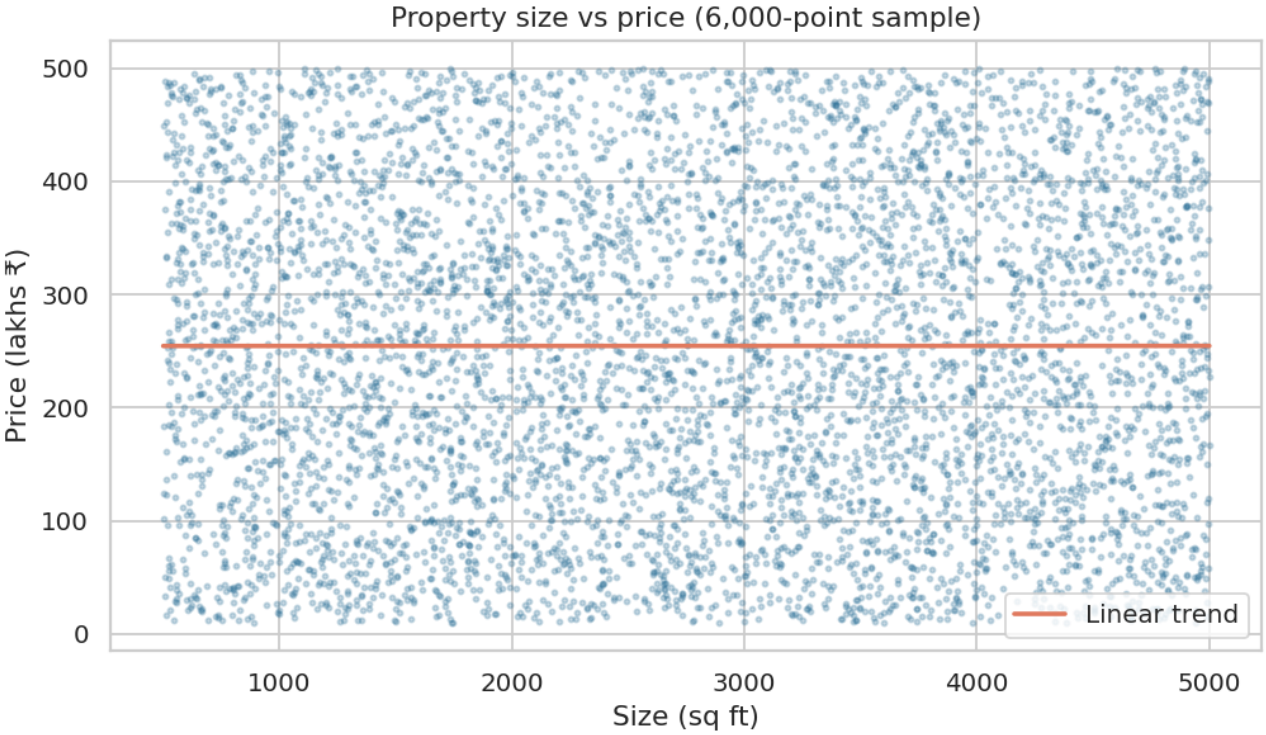
Why keep outliers? They are valid small-home/high-price combinations; they were flagged and winsorised for modelling rather than deleted.

Appendix: selected charts

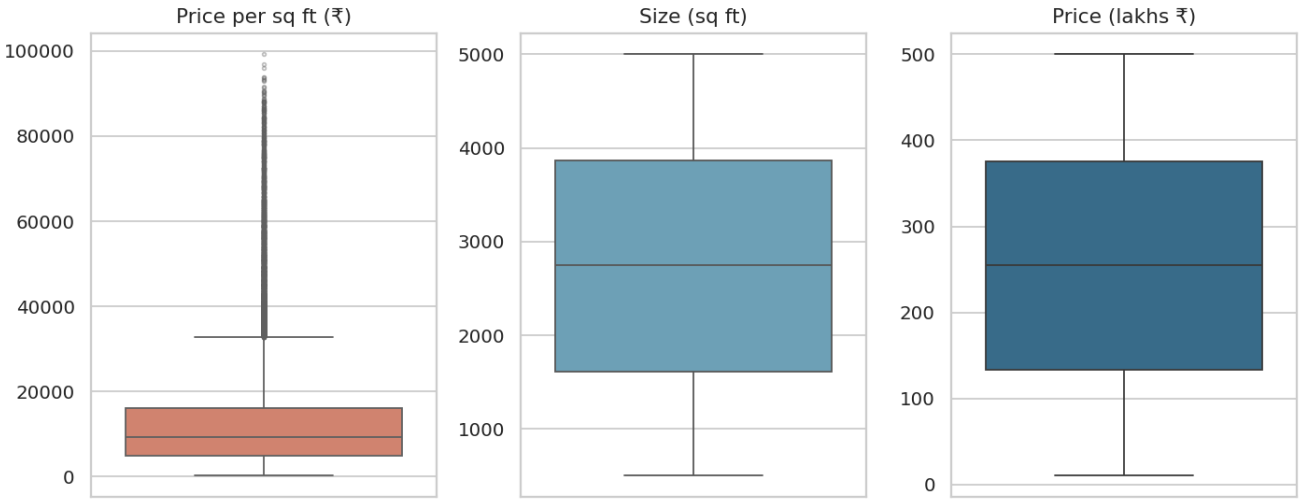
All 24 EDA charts are in `outputs/eda/`; the Streamlit app's EDA page shows them interactively with their tested findings.



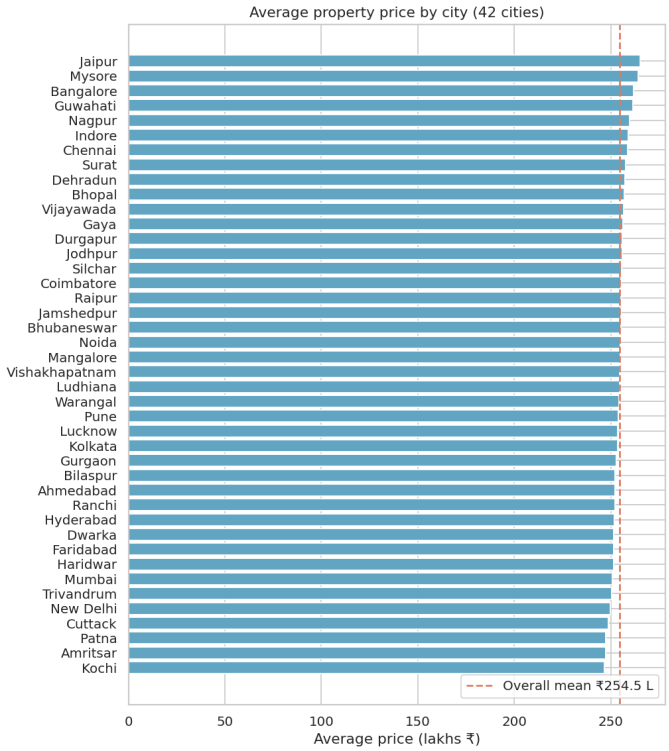
Q1. Price distribution: flat (uniform), not right-skewed like real housing prices.



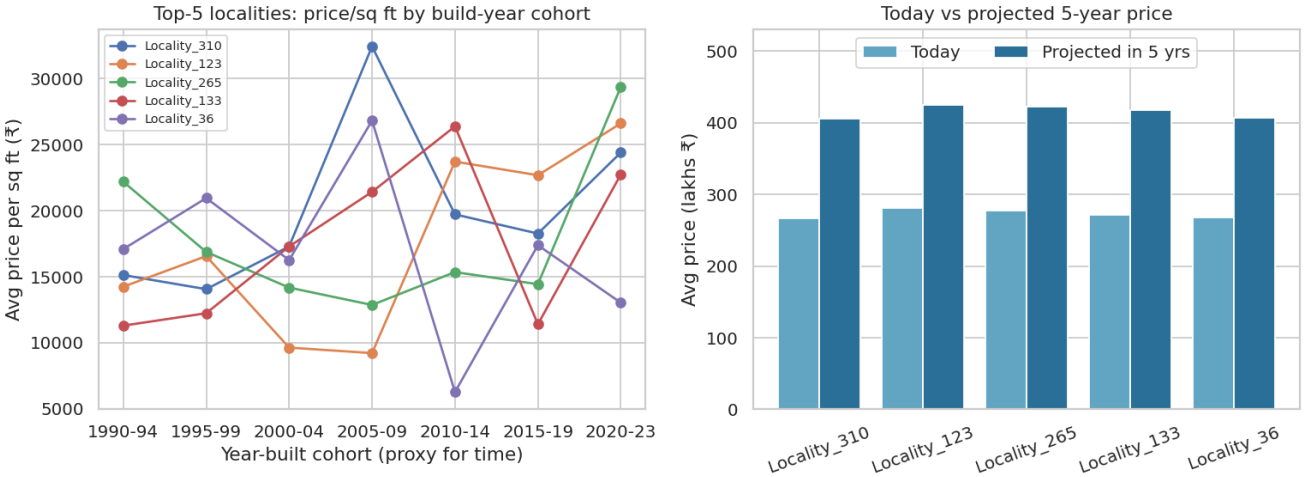
Q4. Size vs price: no relationship (r = 0.000).



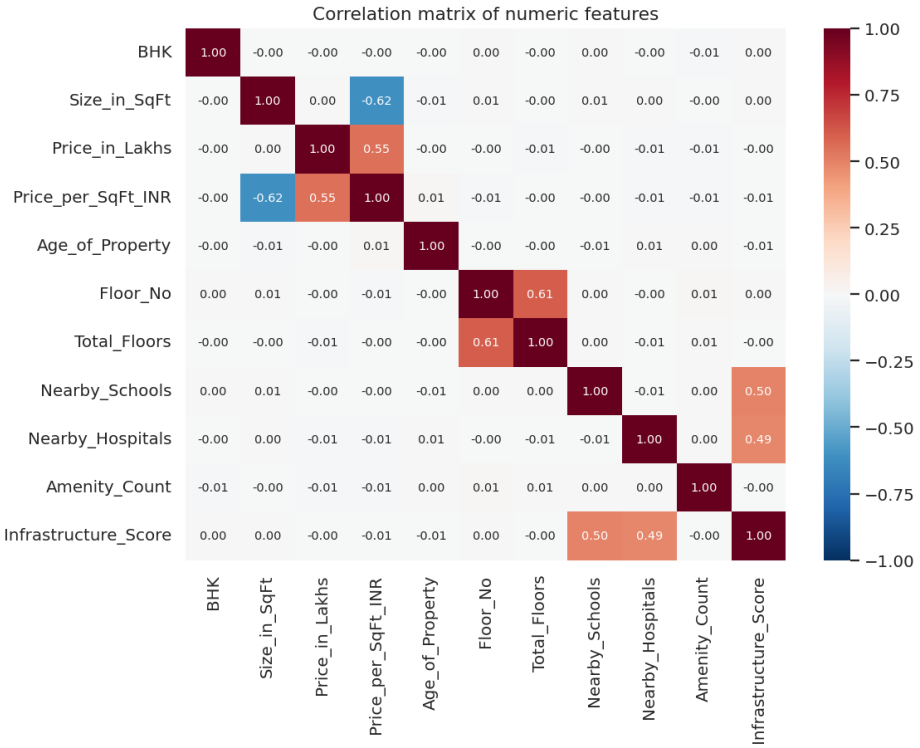
Q5. Outliers: only price per sq ft has outliers (small homes).



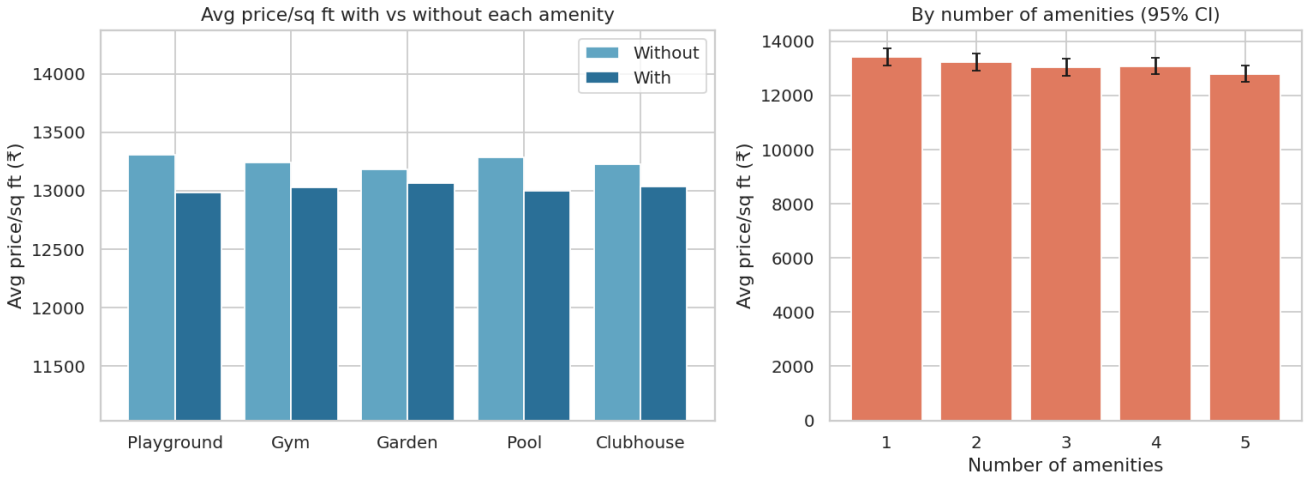
Q7. Average price by city: differences are within noise.



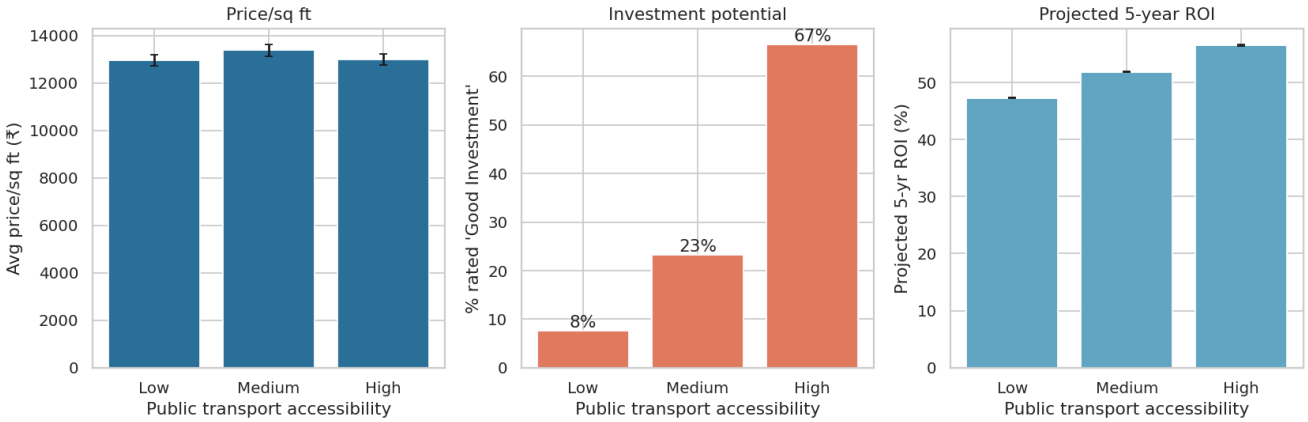
Q10. Top-5 localities: price by build-year cohort, and today vs 5-year projection.



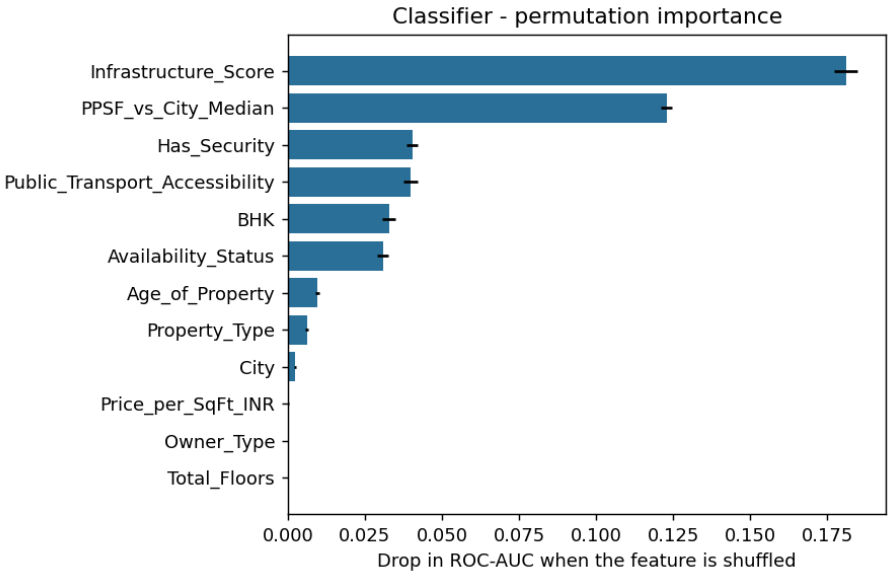
Q11. Correlation matrix: only built-in (arithmetic) links are strong.



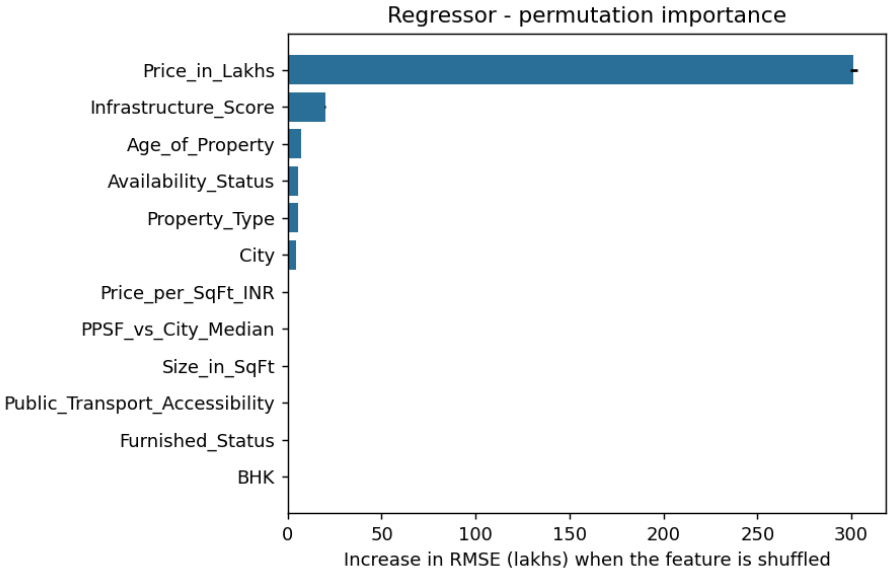
Q19. Amenities vs price per sq ft: negligible differences.



Q20. Public transport: no price effect, but investment potential rises by design of the rule.



Feature importance: Good-Investment classifier (permutation importance, test set).



Feature importance: 5-year price regressor.